Chapter 12

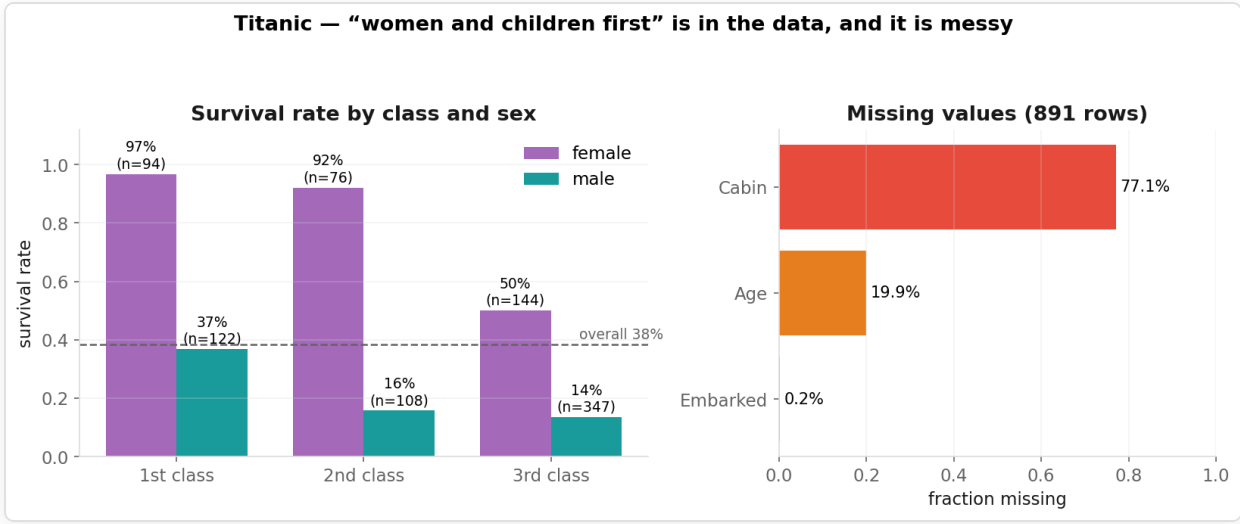
Capstone: End-to-End ML Workflow

Session 4 | Final Chapter Putting it all together

Today's Mission

April 15, 1912. RMS Titanic strikes an iceberg. 1,502 of 2,224 people on board die.

Can we predict who survived — from passenger characteristics?



The Dataset (Kaggle Titanic, `0-datasets/titanic.csv`, 891 rows)

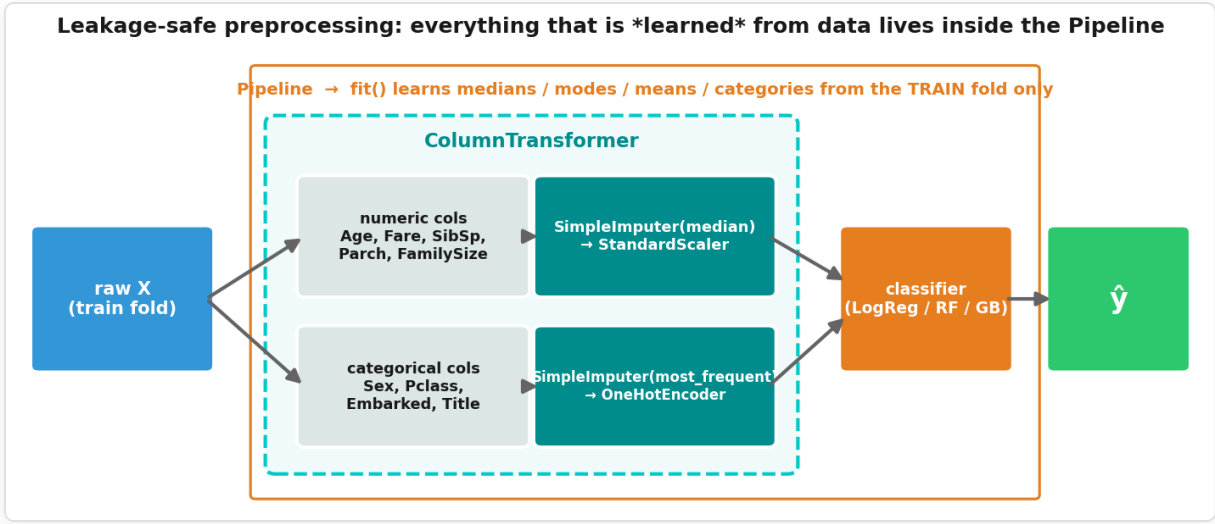
Column	Type	Notes
`Pclass`	ordinal 1/2/3	ticket class — socio-economic proxy
`Sex`	categorical	
`Age`	numeric	**20 % missing** → impute (inside the pipeline!)
`SibSp`, `Parch`	numeric	siblings/spouses, parents/children aboard → `FamilySize`
`Fare`	numeric	ticket price
`Embarked`	categorical C/Q/S	2 missing
`Name`	text	→ extract `Title` (Mr / Mrs / Miss / Master / Rare)
`Cabin`, `Ticket`, `PassengerId`		77 % missing / noisy / id → **drop**

Target: `Survived` (1 = yes). 38 % survived → majority baseline = 62 % accuracy, F1 = 0.

The Plan — and where each chapter shows up

Step	What	Chapter
① Load & explore	EDA is given — read it	Ch01
② **Split first**	`train_test_split(stratify=y)`, test set locked away	Ch02
③ Feature engineering	`FamilySize`, `IsAlone`, `Title` from `Name` (row-wise, no fitting)	Ch02
④ **Preprocessing pipeline**	`SimpleImputer` + `StandardScaler` / `OneHotEncoder` in a `ColumnTransformer`	Ch02
⑤ Baseline + 3 models	`DummyClassifier`, LogReg, RandomForest, GradientBoosting — 5-fold CV	Ch03–06
⑥ Final test evaluation	best-by-CV model, **one** look at the test set, report + confusion matrix	Ch06
⑦ Interpret & reflect	permutation importance, error analysis, fairness question	Ch06
Bonus	`GridSearchCV`, `joblib`, PCA of the passengers	Ch09

Leakage-safe preprocessing — the core of today



`cross_validate(pipe, X_train, y_train, cv=5)` re-fits imputers, scaler and encoder in **every fold** on the training part only.

One new model: Gradient Boosting in one slide

- Random Forest (Ch05): many deep trees **in parallel**, each on a bootstrap sample → average
- **Gradient Boosting:** many *shallow* trees **in sequence**; each new tree fits the **errors** of the previous ones
- Strong default on tabular data; more sensitive to hyperparameters than a forest
- `GradientBoostingClassifier(random_state=42)` — same `fit / predict` interface as everything else

What does "success" mean here?

- We want to **find the survivors** → recall for class 1
- and to be **right when we say "survived"** → precision for class 1
- **F1** balances both → primary metric; accuracy reported alongside

A **false negative** = predicted "died", actually survived.

Rules of the game (50 min): work in order · TODO cells are yours, ▶ cells are given · use the ▶ check cells · bonus only if done · solutions afterwards.

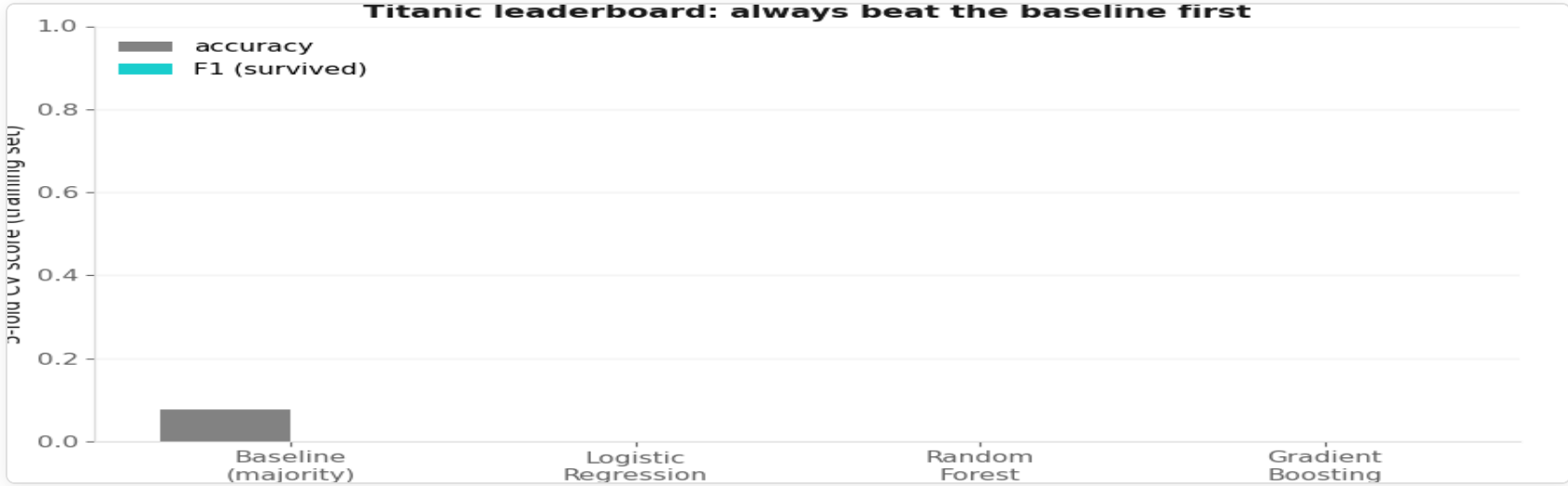
Now — open the notebook

`03-exercises/ch12_capstone_exercises.ipynb`

50 minutes. Split first → features → pipeline → baseline → CV → one test evaluation → interpret.

You've got this.

Debrief (1/3): the leaderboard



Poll: which model won on *your* CV? What F1 on the test set?

Debrief (2/3): what did the model learn?

- **Permutation importance:** `Sex` and `Title` on top, then `Pclass` / `Fare` (compare with the EDA plot!)
- Impurity-based importances (Ch05) would have ranked `Age` / `Fare` higher — continuous columns get an unfair bonus. Permutation importance is the honest version.
- **Error analysis:** the misclassified passengers are the "surprising" cases — 3rd-class men who survived, 1st-class women who did not. No feature we have explains them.
- Models are close → **features + clean evaluation** matter more than the algorithm.

Debrief (3/3): fairness — the uncomfortable question

- The best signal is `Sex` — and `Title` is a proxy for it (plus age and status).
- Predicting **history** is fine. Using the same recipe **today** (loans, triage, insurance) would mean deciding by a protected attribute — directly or via proxies.
- Before deployment you would check: error rates **per group**, proxies for protected attributes, whether the target itself encodes past discrimination.

Question to the room: which of *your* features are proxies for something you would not be allowed to use?

What We've Covered

Chapter	Topic — and where you used it today
Ch01	the workflow — **this is it**
Ch02	split first, impute/encode/scale inside a Pipeline — **the core of today**
Ch03–06	fit / predict, baseline, cross-validation, F1, confusion matrix
Ch07–09	(bonus) PCA of the passengers
Ch10–11	a different paradigm — learning from rewards

You Can Now

- Load and explore **any** tabular dataset
- Split first and preprocess **without leakage** — as a Pipeline
- Beat a baseline, compare models honestly with cross-validation
- Evaluate once on the test set and read the confusion matrix
- Explain what a model learned — and ask whether it *should* have learned it
- Explain what reinforcement learning is and how Q-learning fills its table

What Comes Next · Keep Going

- **Hyperparameter tuning** — `GridSearchCV` / `RandomizedSearchCV` / Optuna (bonus A today)
- **Deep learning** — neural networks for images, text, audio
- **Deployment & MLOps** — `joblib`-saved pipelines (bonus B), monitoring, retraining
- **Practice:** Kaggle (start with this very dataset), fast.ai, Géron *Hands-On ML*

Thank you

Applied Machine Learning — done.

"Split first. Beat the baseline. Evaluate once. Ask what the model learned."